

Rate Limiter - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Before handling an API call, ask: has this user used too many requests? Allow or block.

Story - how this LLD works

FREE users get a Token Bucket (burst-friendly, ~10 per minute in demo). PREMIUM users get a Fixed Window (100 per minute). Service picks the limiter by tier. Under the hood each algorithm keeps per-user state with a Lock.

Who does what

RateLimiterService: Public door: allowRequest(user) -> True/False.

TokenBucket: Tokens refill over time. Empty bucket = blocked. Allows short bursts.

FixedWindow: Count requests in the current time window; reset when window id changes.

SlidingWindowLog: Remember timestamps; drop old ones; count what's inside the window.

RateLimiterFactory: Build the right algorithm from an enum + config.

User + tier: Who is calling and which plan they are on.

Main pieces

Service

- tier -> limiter
- allowRequest

Token Bucket

- refill
- burst OK

Fixed Window

- count / window
- edge burst risk

Sliding Log

- timestamp list
- smoother

Step-by-step at runtime

1. API receives request for user U.
2. Service looks at U.tier -> picks limiter.
3. Limiter.allowRequest(U.id) under lock.
4. Algorithm says yes -> process API. No -> return 429-style block.
5. main.py fires 20 threads at once on FREE tier - only ~10 pass token bucket.

Why this design (plain words)

Why Strategy? - Same allowRequest API, different math per plan.

Why Factory? - Create limiters without scattering if/else in the service.

Why Lock? - Counters/tokens must not corrupt under concurrent requests.

Remember

- Fixed window can burst at boundaries (end of W1 + start of W2).
- In-memory only - multi-server needs Redis for shared limits.
- Some enum algos are listed but not implemented yet - good practice task.

OOP in this project

Encapsulation - Token maps stay private inside the limiter.

Abstraction - Caller only knows allowRequest.

Inheritance - Each algorithm extends RateLimiter.

Polymorphism - Service calls allowRequest on whatever algorithm tier mapped to.

DESIGN.md - full file

Complete markdown from lld/rate_limiter/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Rate Limiter - LLD Q&A

Run: `python -m lld.rate_limiter.main`

Patterns: Strategy · Factory · Facade/Service · Thread safety

Q1. What problem are we solving?

A. Design an API rate limiter that:

Enforces per-user request quotas by subscription tier

Supports multiple algorithms (Token Bucket, Fixed Window, Sliding Window Log)

Is thread-safe under concurrent requests

Returns allow / block for each request

Q2. What are the core entities?

- Entity - Responsibility
- User - Identity + tier (FREE / PREMIUM)
- RateLimitConfig - max_requests, window_in_seconds
- RateLimiter (ABC) - allow_request(user_id) -> bool
- RateLimiterFactory - Algo enum -> concrete limiter
- RateLimiterService - Maps tier -> limiter; public API
- Demo mapping:

Q3. How is the package structured?

```
rate_limiter/
|-- model/      # User, RateLimitConfig
|-- limiter/    # TokenBucket, FixedWindow, SlidingWindowLog
|-- factory/    # RateLimiterFactory
|-- service/    # RateLimiterService
|-- enums/     # RateLimitType, UserTier
```

Q4. Why Strategy for algorithms?

A. Algorithms differ; callers only need allow_request. Swap without changing the service API.

| Algorithm | Idea |

|-----|-----|

| Token Bucket | Tokens refill over time; allows short bursts |

| Fixed Window | Count requests per fixed time window |

| Sliding Window Log | Store timestamps; prune expired; count in window |

Each implementation uses threading.Lock on its per-user state.

Q5. Why Factory?

A. RateLimiterFactory.create_rate_limiter(algo, config) maps enum -> class. Service does not hard-code constructors.

Q6. What is the request flow?

```
RateLimiterService.allow_request(user)
-> pick limiter by user.tier
-> limiter.allow_request(user_id) # under lock
-> True (ALLOWED) / False (BLOCKED)
```

main.py stress-tests with 20 concurrent threads -> ~10 allowed for FREE token bucket.

Q7. How do the algorithms compare?

- Token Bucket - Fixed Window | Sliding Window Log
- Burst - Yes | At window edges | Smooth
- Memory - Low (tokens + last refill) | Low (count + window id) | Higher (timestamp list)
- Accuracy - Good | Boundary burst problem | Most accurate

Q8. How do you extend this design?

- Add... - How
- Leaky Bucket / Sliding Counter - New limiter class + factory case (enums already hint at these)
- Per-endpoint limits - Key = user_id + endpoint
- Distributed limiting - Redis INCR/EXPIRE or Redis cell
- Retry metadata - Return remaining, retry_after instead of bool

Q9. Common interview follow-ups

Q. Fixed window boundary burst?

User can send max at end of window W1 and max at start of W2 -> 2x limit in a short span. Sliding window fixes this.

Q. Token bucket refill rate in this code?

window / max_requests seconds per token (inverted from "tokens per second" wording - know both formulations).

Q. Multi-server?

In-memory state is per process - need Redis (or similar) for cluster-wide limits.

Source code - lld/rate_limiter/

11 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (59 lines)

```

1| import concurrent.futures
2| import threading
3|
4| from .enums.user_tier import UserTier
5| from .model.user import User
6| from .service.rate_limiter_service import RateLimiterService
7|
8|
9| def check_concurrency(rate_limiter_service: RateLimiterService) -> None:
10|     free_user1 = User("user1", UserTier.FREE)
11|
12|     threads = 20
13|     barrier = threading.Barrier(threads)
14|     remaining = threads
15|     remaining_lock = threading.Lock()
16|     latch_event = threading.Event()
17|
18|     def count_down() -> None:
19|         nonlocal remaining
20|         with remaining_lock:
21|             remaining -= 1
22|             if remaining == 0:
23|                 latch_event.set()
24|
25|     with concurrent.futures.ThreadPoolExecutor(max_workers=threads) as executor:
26|         for i in range(1, threads + 1):
27|             req_num = i
28|
29|             def task(req_num: int = req_num) -> None:
30|                 try:
31|                     barrier.wait()
32|                 except Exception:
33|                     import traceback
34|
35|                     traceback.print_exc()
36|
37|                 allowed = rate_limiter_service.allow_request(free_user1)
38|                 print(
39|                     f'{threading.current_thread().name} | Request {req_num} "
40|                     f"for FreeUser1: {'ALLOWED' if allowed else 'BLOCKED'}"
41|                 )
42|                 count_down()
43|
44|             executor.submit(task)
45|
46|     latch_event.wait()
47|
48|
49| def main() -> None:
50|     rate_limiter_service = RateLimiterService()
51|
52|     # free_user = User("user1", UserTier.FREE)
53|     # premium_user = User("user2", UserTier.PREMIUM)
54|
55|     check_concurrency(rate_limiter_service)
56|
57|
58| if __name__ == "__main__":
59|     main()

```

FILE: service/rate_limiter_service.py (26 lines)

```

1| from ..enums.rate_limit_type import RateLimitType
2| from ..enums.user_tier import UserTier
3| from ..factory.rate_limiter_factory import RateLimiterFactory
4| from ..limiter.rate_limiter import RateLimiter
5| from ..model.rate_limit_config import RateLimitConfig
6| from ..model.user import User
7|
8|
9| class RateLimiterService:
10|     def __init__(self) -> None:
11|         self._rate_limiters: dict[UserTier, RateLimiter] = {
12|             UserTier.FREE: RateLimiterFactory.create_rate_limiter(
13|                 RateLimitType.TOKEN_BUCKET,
14|                 RateLimitConfig(max_requests=10, window_in_seconds=60),
15|             ),
16|             UserTier.PREMIUM: RateLimiterFactory.create_rate_limiter(
17|                 RateLimitType.FIXED_WINDOW,
18|                 RateLimitConfig(max_requests=100, window_in_seconds=60),
19|             ),
20|         }
21|
22|     def allow_request(self, user: User) -> bool:
23|         limiter = self._rate_limiters.get(user.tier)
24|         if limiter is None:
25|             raise ValueError(f"No limiter configured for tier: {user.tier}")
26|         return limiter.allow_request(user.user_id)

```

FILE: model/rate_limit_config.py (7 lines)

```

1| from dataclasses import dataclass
2|
3|
4| @dataclass
5| class RateLimitConfig:
6|     max_requests: int
7|     window_in_seconds: int

```

FILE: model/user.py (9 lines)

```

1| from dataclasses import dataclass
2|
3| from ..enums.user_tier import UserTier
4|
5|
6| @dataclass
7| class User:
8|     user_id: str
9|     tier: UserTier

```

FILE: limiter/fixed_window_rate_limiter.py (35 lines)

```

1| import threading
2| import time
3|
4| from ..enums.rate_limit_type import RateLimitType
5| from ..model.rate_limit_config import RateLimitConfig
6| from .rate_limiter import RateLimiter
7|
8|
9| class FixedWindowRateLimiter(RateLimiter):
10|     def __init__(self, config: RateLimitConfig) -> None:
11|         super().__init__(config, RateLimitType.FIXED_WINDOW)
12|         self._request_count: dict[str, int] = {}
13|         self._window_start: dict[str, int] = {}
14|         self._request_count_lock = threading.Lock()
15|
16|     def allow_request(self, user_id: str) -> bool:
17|         allowed = False
18|         current_req_window = int(time.time()) // self.config.window_in_seconds
19|
20|         with self._request_count_lock:
21|             last_req_window = self._window_start.get(user_id, current_req_window)
22|
23|             if last_req_window != current_req_window:
24|                 self._window_start[user_id] = current_req_window
25|                 allowed = True
26|                 self._request_count[user_id] = 1
27|             else:
28|                 count = self._request_count.get(user_id, 0)
29|                 if count < self.config.max_requests:
30|                     allowed = True
31|                     self._request_count[user_id] = count + 1
32|                 else:
33|                     self._request_count[user_id] = count
34|
35|         return allowed

```

FILE: limiter/rate_limiter.py (14 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ..enums.rate_limit_type import RateLimitType
4| from ..model.rate_limit_config import RateLimitConfig
5|
6|
7| class RateLimiter(ABC):
8|     def __init__(self, config: RateLimitConfig, type_: RateLimitType) -> None:
9|         self.config = config
10|         self.type = type_
11|
12|     @abstractmethod
13|     def allow_request(self, user_id: str) -> bool:
14|         pass

```

FILE: limiter/sliding_window_log_rate_limiter.py (34 lines)

```

1| import threading
2| import time
3| from collections import deque
4|
5| from ..enums.rate_limit_type import RateLimitType
6| from ..model.rate_limit_config import RateLimitConfig
7| from .rate_limiter import RateLimiter
8|
9|
10| class SlidingWindowLogRateLimiter(RateLimiter):
11|     def __init__(self, config: RateLimitConfig) -> None:
12|         super().__init__(config, RateLimitType.SLIDING_WINDOW_LOG)
13|         self._request_log: dict[str, deque[int]] = {}
14|         self._request_log_lock = threading.Lock()
15|
16|     def allow_request(self, user_id: str) -> bool:
17|         allowed = False
18|         now = int(time.time())
19|
20|         with self._request_log_lock:
21|             log = self._request_log.get(user_id)
22|             if log is None:
23|                 log = deque()
24|

```

```

25|         while log and (now - log[0]) >= self.config.window_in_seconds:
26|             log.popleft()
27|
28|         if len(log) < self.config.max_requests:
29|             log.append(now)
30|             allowed = True
31|
32|         self._request_log[user_id] = log
33|
34|     return allowed

```

FILE: limiter/token_bucket_rate_limiter.py (43 lines)

```

1| import threading
2| import time
3|
4| from ..enums.rate_limit_type import RateLimitType
5| from ..model.rate_limit_config import RateLimitConfig
6| from .rate_limiter import RateLimiter
7|
8|
9| class TokenBucketRateLimiter(RateLimiter):
10|     def __init__(self, config: RateLimitConfig) -> None:
11|         super().__init__(config, RateLimitType.TOKEN_BUCKET)
12|         self._tokens: dict[str, int] = {}
13|         self._last_refill_time: dict[str, int] = {}
14|         self._tokens_lock = threading.Lock()
15|
16|     def allow_request(self, user_id: str) -> bool:
17|         allowed = False
18|         now = int(time.time()) * 1000
19|
20|         with self._tokens_lock:
21|             current_tokens = self._refill_tokens(user_id, now)
22|
23|             if current_tokens > 0:
24|                 allowed = True
25|                 self._tokens[user_id] = current_tokens - 1
26|             else:
27|                 self._tokens[user_id] = current_tokens
28|
29|         return allowed
30|
31|     def _refill_tokens(self, user_id: str, now: int) -> int:
32|         refill_rate = self.config.window_in_seconds / self.config.max_requests
33|
34|         last_refill = self._last_refill_time.get(user_id, now)
35|         elapsed_seconds = (now - last_refill) // 1000
36|         refill_tokens = int(elapsed_seconds / refill_rate)
37|
38|         current_tokens = self._tokens.get(user_id, self.config.max_requests)
39|         current_tokens = min(self.config.max_requests, current_tokens + refill_tokens)
40|         if refill_tokens > 0:
41|             self._last_refill_time[user_id] = now
42|
43|         return current_tokens

```

FILE: factory/rate_limiter_factory.py (18 lines)

```

1| from ..enums.rate_limit_type import RateLimitType
2| from ..limiter.fixed_window_rate_limiter import FixedWindowRateLimiter
3| from ..limiter.rate_limiter import RateLimiter
4| from ..limiter.sliding_window_log_rate_limiter import SlidingWindowLogRateLimiter
5| from ..limiter.token_bucket_rate_limiter import TokenBucketRateLimiter
6| from ..model.rate_limit_config import RateLimitConfig
7|
8|
9| class RateLimiterFactory:
10|     @staticmethod
11|     def create_rate_limiter(algo: RateLimitType, config: RateLimitConfig) -> RateLimiter:
12|         if algo == RateLimitType.TOKEN_BUCKET:
13|             return TokenBucketRateLimiter(config)
14|         if algo == RateLimitType.FIXED_WINDOW:
15|             return FixedWindowRateLimiter(config)
16|         if algo == RateLimitType.SLIDING_WINDOW_LOG:
17|             return SlidingWindowLogRateLimiter(config)
18|         raise ValueError(f"Unknown algorithm: {algo}")

```

FILE: enums/rate_limit_type.py (9 lines)

```

1| from enum import Enum
2|
3|
4| class RateLimitType(Enum):
5|     TOKEN_BUCKET = "TOKEN_BUCKET"
6|     LEAKY_BUCKET = "LEAKY_BUCKET"
7|     FIXED_WINDOW = "FIXED_WINDOW"
8|     SLIDING_WINDOW_LOG = "SLIDING_WINDOW_LOG"
9|     SLIDING_WINDOW_COUNTER = "SLIDING_WINDOW_COUNTER"

```

FILE: enums/user_tier.py (6 lines)

```

1| from enum import Enum
2|
3|
4| class UserTier(Enum):
5|     FREE = "FREE"
6|     PREMIUM = "PREMIUM"

```